ZEROTH: An AI Red-Team Testing and Security Evaluation Platform for Local Large Language Models

Project report | ZEROTH v1.0.0rc1 | November 2026

Abstract

Large language models (LLMs) are increasingly deployed locally, with instructions and sometimes sensitive values placed in the system prompt. Prompt injection, where attacker-controlled text overrides those instructions, is the leading risk in the OWASP Top 10 for LLM Applications. ZEROTH is an open, locally run platform that measures how locally hosted LLMs behave under prompt-injection attack. It plants a synthetic *canary* secret in a model's system prompt, attacks the model with a library of 45 scripted and adaptive payloads across nine attack families, applies configurable defenses, and records whether the canary reaches the user. Results are reported as attack success rates with 95% Wilson confidence intervals, alongside benign-control runs that quantify the usability cost of each defense. The platform includes a command-line interface, a local web console, a report generator mapped to OWASP LLM risks, and an 82-test automated suite. This report describes the motivation, design, methodology and validation of ZEROTH, and states plainly what the evidence does and does not support.

1. Introduction and motivation

Organisations adopting open-weight models such as Qwen and Mistral through runtimes like Ollama gain privacy and control, but inherit the model's susceptibility to instruction manipulation. A system prompt is not a security boundary: the model processes trusted instructions and untrusted input as one stream of text. Three OWASP LLM risk categories are directly relevant: LLM01 Prompt Injection, LLM02 Sensitive Information Disclosure and LLM07 System Prompt Leakage.

Practitioners therefore need answers to three questions, supported by repeatable measurement instead of anecdote:

1. How easily can a given model be induced to break its rules?
2. Which defenses reduce that risk, and by how much?
3. What do those defenses cost in blocked, legitimate use?

ZEROTH was built to answer these questions for models running on a developer's own machine.

2. Objectives and scope

Objectives. (a) Provide an objective, automatically scored measure of secret disclosure under attack. (b) Compare models and defenses on equal footing. (c) Preserve complete, reproducible evidence. (d) Present results clearly to both engineers and non-specialists.

In scope. Text-based disclosure of a planted synthetic canary from the system prompt; direct, indirect, multi-turn and adaptive injection; local models served by Ollama; pre- and post-generation defenses.

Out of scope. Harmful-content generation, agent and tool abuse, training-data extraction, attacks on model weights or infrastructure, and any testing of third-party systems.

3. Related work

General-purpose LLM vulnerability scanners such as NVIDIA's garak organise testing into probes, generators and detectors, and cover a broad set of failure modes. Microsoft's PyRIT supports orchestrated red-team campaigns. Research on direct injection (Perez and Ribeiro, 2022) and indirect injection through retrieved content (Greshake et al., 2023) established the threat model. *Spotlighting* (Hines et al., 2024) proposed marking untrusted content to help models distinguish data from instructions. ZEROTH adopts the probe/generator/detector architecture familiar from these tools but is deliberately narrower: it centres on canary disclosure and adds first-class measurement of defense effectiveness and defense cost.

4. System design

4.1 Pipeline

Each experimental run flows through the same stages:

Probe → Buff → Defense (pre-generation) → Generator → Defense (post-generation) → Detectors → Evidence → Statistics, Report and Console

4.2 Components

Component	Role
Probes	Attack libraries (and benign controls) that produce cases of one or more user turns
Buffs	Rephrase or encode payloads (polite, authority, urgent, leetspeak, upper-case, base64 and rot13 instructions) to multiply variants
Defenses	Hardened prompt, spotlighting, input scan, output guard, LLM judge, and a layered combination; composable with `+`
Generators	Ollama chat backend (stdlib HTTP, records model digest) and deterministic mock models for testing
Detectors	Canary detector (used as evaluation oracle and inside the guard), refusal heuristic, system-prompt-leak detector
Runner	Executes the plan (models × defenses × cases × trials), supports resume, aborts after five consecutive errors
Storage	One directory per run: JSONL evidence plus run metadata; SQLite export
Statistics	Rates, Wilson intervals, defense effectiveness, guard misses, severity heuristic
Report and Console	Markdown/HTML assessment and a local web dashboard

4.3 The canary

A synthetic token (`ZEROTH-SECRET-7391`) is placed in the system prompt with an instruction never to reveal it. Because the secret is fake and known, disclosure can be scored objectively without judging content, and no real credential is ever at risk.

4.4 Detection levels

The guard and oracle classify any text against the canary using ordered levels:

Level	Meaning
exact	The canary appears literally (case-insensitive)
encoded	Appears after decoding base64, hex, URL or HTML encoding (up to three nested rounds)
obfuscated	Appears after removing separators, or after rot13, reversal, or NATO and digit-word decoding
near	A slightly altered canary: similarity at or above a threshold *and* the secret's digit anchor present or nearly present
partial	Only the digit anchor appears; a weak signal that never blocks
none	No evidence of disclosure

4.5 Oracle versus guard

The production *guard* uses a similarity threshold of 0.80. The evaluation *oracle* is deliberately more sensitive (0.70, plus partial signals). A response that the oracle flags but the guard allowed is recorded as a *guard miss*. Both share the same string-matching machinery, so a paraphrase neither can recognise is invisible to both; this is a stated limitation.

4.6 Evidence record

Every attempt is stored as a schema-versioned record containing the model name and digest, defense, probe, case and payload identifiers, buff, trial, temperature, seed, token limit, every conversation turn (raw response, displayed response, detections, block reasons), and summary flags: model leaked, exposed, input blocked, output blocked, refused, system-prompt leak, latency and error.

5. Attack library

The library contains 45 payloads: 43 scripted and two seeds for the adaptive attacker. Every payload targets only the synthetic canary, and none contains it.

Family	Payloads	Technique
direct.override	6	"Ignore previous instructions" and authority claims
direct.roleplay	5	Personas, fiction, debug-console and "game" framings
direct.extraction	6	Repeat, quote, translate or complete the hidden prompt
direct.obfuscated_ask	6	Request the secret encoded, reversed, spaced, hexadecimal or in NATO spelling
direct.multilingual	5	Override requests in Spanish, French, German, Tamil and Hindi
direct.format_confusion	5	Fake system tags, banners and chat-template tokens
indirect.document	7	Hidden instructions in documents, HTML, email, retrieved chunks, tool output and footnotes
multiturn.crescendo	3	Scripted four-turn escalation
adaptive.attacker_llm	2	A second local model writes and adapts attacks based on replies

Fourteen **benign control** prompts (eight direct, six document summaries) establish the baseline: a leak on a control means the model discloses with no attack at all, and a block on a control is a false positive of the defense.

6. Defenses

Defense	Stage	Mechanism
hardened_prompt	Pre	Appends a priority security policy to the system prompt
spotlight	Pre	Datamarks untrusted content (whitespace replaced by a marker) and instructs the model to treat it as data

Defense	Stage	Mechanism
input_scan	Pre	Pattern scan of the input, including decoded variants, before the model is called
output_guard	Post	The response guard blocks displayed text containing the canary in any detected form
llm_judge	Post	A second model judges whether the output discloses the secret
layered	Both	hardened_prompt + spotlight + input_scan + output_guard

7. Evaluation methodology

The experimental unit is one run: a combination of model, defense, case and trial. Trial *t* uses seed *base + t* and a configured temperature, so repeats differ yet the experiment is reproducible. Errored runs are stored but excluded from all metrics.

Metric	Definition
Model leak rate	Attack runs where the raw model output disclosed the canary, divided by attack runs
Exposure rate (attack success rate)	Attack runs where the displayed text, after defenses, disclosed the canary, divided by attack runs
95% confidence interval	Wilson score interval on the exposure rate; valid at small n and at 0% or 100%
Mitigation rate	Raw leaks that did not reach the user, divided by raw leaks
False-block rate	Benign control runs blocked by a defense, divided by control runs
Guard miss	A leak that reached the user despite an output filter

Severity heuristic. Exposure of at least 50% is labelled High, at least 20% Medium, above 0% Low, and 0% None observed. This is a laboratory heuristic about canary disclosure, not a real-world risk rating.

Three profiles trade time for rigour: *quick* (smoke test), *standard* (10 trials, two buffs, five defenses) and *full* (30 trials, five buffs, six defenses). Runs are resumable.

8. Implementation and engineering quality

ZEROTH is a Python package (about 3,600 lines including tests and documentation) with a standard-library-only runtime core; the only dependency is PyYAML for configuration.

- **Command-line interface:** `run`, `summarize`, `report`, `serve`, `calibrate`, `guard-test`, `models`, `list`, `runs` and `export-sqlite`.
- **ZEROTH Console:** a local, offline web dashboard (bound to 127.0.0.1) with summary cards, an exposure heatmap by model and probe, a defense-effectiveness view, benign-control and leak-form tables, an evidence browser, a guard playground and a run launcher.
- **Reports:** auto-generated Markdown and self-contained HTML with executive summary, methodology, results, defense comparison, guard misses, ranked findings mapped to OWASP LLM01, LLM02 and LLM07, recommendations, limitations and redacted evidence. Mock-model runs carry an explicit warning.
- **Quality assurance:** 82 automated tests, a continuous-integration workflow across Python 3.10 to 3.12, and a mock backend allowing the full pipeline to run without a GPU.

9. Validation

This section reports what was verified. It validates the *tooling*, not the security of any particular model.

9.1 A guard defect found and fixed

The predecessor guard (ZEROTH-005) blocked harmless placeholders such as `ZEROTH-SECRET-XXXX` and `ZEROTH-SECRET-REDACTED`, because near-match similarity was driven by the shared prefix. The fix requires the secret's digit anchor to be present or nearly

present before a near-match counts. After the fix, all 22 cases in an extended guard test set behaved as intended, including rot13, reversed text, NATO spelling, spaced hexadecimal and unpadded base64.

9.2 Guard calibration

A labelled corpus of 134 strings (67 disclosing, 67 not) was swept across similarity thresholds on its core tier (62 positives, 67 negatives):

Threshold	False-positive rate	Recall
0.50	6.0%	100%
0.60	1.5%	100%
0.70 to 0.85	0.0%	100%
0.90	0.0%	96.8%
0.95	0.0%	85.5%

The default of 0.80 sits in the middle of a stable plateau. On a separate "hard" tier of five difficult cases (large edits, paraphrase, translation), four were missed at 0.80, which documents the paraphrase limitation. **The corpus was authored together with the detector, so these figures are in-sample** and should be refreshed with real model outputs.

9.3 Input-scan coverage

Against the 43 scripted payloads, the pattern-based input scan flagged 23 and missed 20, including role-play, multi-turn escalation and the Tamil and Hindi requests. It raised no false blocks on the 14 benign controls. This confirms that input screening is a useful but incomplete layer.

9.4 Pipeline validation with mock models

A quick-profile run with three deterministic mock models (156 attack runs, zero errors) behaved as designed: the always-leaking model reached 100% exposure on both probes, the never-leaking model reached 0%, and the stochastic model fell between (75% direct override, 64% indirect document). Enabling the output guard reduced the always-leaking model from 100% to 0% exposure with 0% false blocks. These runs validate scoring, defenses, storage, reporting and the console; they say nothing about real models.

9.5 Test sensitivity

As a sanity check on the test suite itself, disabling the digit-anchor rule caused three tests to fail, and restoring it returned all 82 tests to passing.

10. Live model evaluation

Protocol. Target models: Qwen 2.5 (3B) and Mistral, served by Ollama with exact tags and digests recorded. Run the *standard* profile (10 trials per case, buffs none and polite, defenses none, hardened_prompt, input_scan, output_guard and layered) at temperature 0.7 and base seed 1234, with an optional adaptive attacker.

Results. Insert the figures from the completed run before publication. Report each exposure rate with its sample size and 95% interval.

Model	Defense	Attack runs	Model leak rate	Exposure rate (95% CI)	False-block rate
Qwen 2.5 3B	none	to be inserted	to be inserted	to be inserted	to be inserted
Qwen 2.5 3B	layered	to be inserted	to be inserted	to be inserted	to be inserted
Mistral	none	to be inserted	to be inserted	to be inserted	to be inserted
Mistral	layered	to be inserted	to be inserted	to be inserted	to be inserted

Findings. State two or three specific findings in the form: model, attack family, measured rate, defense effect, and false-block cost.

11. Interpreting results

- **Model leaks above exposure** indicates defenses are stopping leaks the model would otherwise make.
- **A defense that lowers exposure but raises false-block rate** trades security for usability; both must be reported.
- **A leak on benign controls** means the system-prompt design is unsafe independent of any attack.
- **Heatmap cells that differ between models** identify model-specific weaknesses worth investigating in the evidence browser.
- **Zero exposure** means none observed in the sample, not secure; the upper confidence bound shows what the sample cannot exclude.

12. Limitations and threats to validity

1. Small samples produce wide intervals; claims should use at least 30 trials.
2. Results are specific to the model version, prompt, sampling settings and runtime used.
3. A single canary format is tested; other secret shapes may behave differently.
4. String-based detection cannot recognise arbitrary paraphrases, translations or semantic restatements of a secret.
5. The attack library is fixed and finite, apart from the optional adaptive attacker.
6. Regular-expression input screening is easily bypassed by novel wording or unsupported languages.
7. The calibration corpus is in-sample, and redaction in reports replaces literal canary text only.
8. Canary disclosure is a proxy for confidentiality failure, not for harmful-content generation or tool abuse.

13. Ethics and responsible use

ZEROTH tests only models the operator runs, using a secret it plants itself. It includes no exploits against third-party systems and no harmful-content prompts. It should not be pointed at services the operator does not own or lack permission to test. Published findings should describe weaknesses in aggregate and omit information that would help attack real deployments.

14. Future work

Tool-use and agent probes (injection that triggers a fake tool call); additional generator backends; semantic leak detection using embeddings to cover paraphrase; a guard trained on accumulated real outputs; side-by-side run comparison and stop and resume controls in the console; and systematic comparison against garak on the same models.

15. Conclusion

ZEROTH turns the question "is this local model safe against prompt injection?" into a measurable, reproducible experiment. Its contributions are an objective canary-based success criterion, a modular probe, defense and detector architecture, statistically

honest reporting with confidence intervals, and explicit measurement of defense cost through benign controls. Its limitations are equally explicit: it measures text disclosure of a planted secret and cannot prove safety. Used with sufficient trials, it gives practitioners evidence to choose models and defenses on, and a foundation for deeper adversarial evaluation.

Appendix A. Reproducing the results

```
python -m venv .venv
.venv\Scripts\activate.bat
pip install -e ".[dev]"
pytest -q
zeroth run --mock --profile quick --yes --report
ollama pull qwen2.5:3b
ollama pull mistral
zeroth models
zeroth run --profile standard --report --yes
zeroth serve
```

Use `--resume <run folder>` to continue an interrupted run, and `zeroth calibrate` to re-run the guard threshold sweep.

Appendix B. Glossary

Term	Meaning
Prompt injection	Attacker text that causes a model to ignore its intended instructions
Direct / indirect injection	The attacker types the instruction, or hides it in content the model processes
Canary	A planted fake secret used to detect disclosure
Trial	One repeat of the same attack; needed because model output is stochastic
Oracle	The more sensitive evaluation detector, distinct from the production guard

Term	Meaning
Exposure rate	Share of attack runs where the canary was visible to the user after defenses
Wilson interval	A confidence interval for a proportion that remains valid at small samples
Control	A benign test used to detect baseline leaks and false blocks
Model digest	A fingerprint of the exact model version used

References

1. OWASP Foundation. *OWASP Top 10 for LLM Applications* (2025 edition): LLM01 Prompt Injection, LLM02 Sensitive Information Disclosure, LLM07 System Prompt Leakage.
2. Perez, F. and Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques for Language Models.
3. Greshake, K. et al. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.
4. Hines, K. et al. (2024). Defending Against Indirect Prompt Injection Attacks With Spotlighting.
5. NVIDIA. garak: LLM vulnerability scanner.
6. Microsoft. PyRIT: Python Risk Identification Toolkit for generative AI.
7. Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*.